

C# OOP Concepts: Classes, Objects, and Constructors

1. Classes and Objects

- ◆ A Class is a blueprint that defines variables (fields) and methods (functions). It doesn't store actual data until an Object is created.
- ◆ An Object is an instance of a Class. Using the 'new' keyword, we create objects that hold real values.

Example 1: Student Class

```
using System;

class Student
{
    public int rollNo;
    public string name;

    public void DisplayInfo()
    {
        Console.WriteLine("Roll No: " + rollNo + ", Name: " + name);
    }
}

class Program
{
    static void Main(string[] args)
    {
        Student s1 = new Student();
        s1.rollNo = 1;
        s1.name = "Saurav";
        s1.DisplayInfo();

        Student s2 = new Student();
        s2.rollNo = 2;
        s2.name = "Ankit";
        s2.DisplayInfo();
    }
}
```

✅ Output:

Roll No: 1, Name: Saurav

Roll No: 2, Name: Ankit

Example 2: Car Class

using System;

```
class Car
{
    public string brand;
    public int speed;

    public void Drive()
    {
        Console.WriteLine(brand + " is driving at " + speed + " km/h");
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Car car1 = new Car();
        car1.brand = "BMW";
        car1.speed = 120;
        car1.Drive();

        Car car2 = new Car();
        car2.brand = "Audi";
        car2.speed = 150;
        car2.Drive();
    }
}
```

✅ Output:

BMW is driving at 120 km/h

Audi is driving at 150 km/h

2. Constructors in C#

◆ A Constructor is a special method in a class with the same name as the class. It is automatically called when an object is created. It is mainly used to initialize fields.

Types of Constructors:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor
4. Static Constructor
5. Private Constructor

2.1 Default Constructor

```
class Student
{
    public int rollNo;
    public string name;

    public Student()
    {
        rollNo = 0;
        name = "Unknown";
    }

    public void Display()
    {
        Console.WriteLine("Roll No: " + rollNo + ", Name: " + name);
    }
}

class Program
{
    static void Main(string[] args)
    {
        Student s1 = new Student();
        s1.Display();
    }
}
```



Output:

Roll No: 0, Name: Unknown

2.2 Parameterized Constructor

```
class Student
{
    public int rollNo;
```

```

public string name;

public Student(int r, string n)
{
    rollNo = r;
    name = n;
}

public void Display()
{
    Console.WriteLine("Roll No: " + rollNo + ", Name: " + name);
}
}

class Program
{
    static void Main(string[] args)
    {
        Student s1 = new Student(1, "Saurav");
        Student s2 = new Student(2, "Ankit");

        s1.Display();
        s2.Display();
    }
}

```

✅ Output:

Roll No: 1, Name: Saurav

Roll No: 2, Name: Ankit

2.3 Copy Constructor

```

class Student
{
    public int rollNo;
    public string name;

    public Student(int r, string n)
    {
        rollNo = r;
        name = n;
    }
}

```

```

public Student(Student s)
{
    rollNo = s.rollNo;
    name = s.name;
}

public void Display()
{
    Console.WriteLine("Roll No: " + rollNo + ", Name: " + name);
}
}

class Program
{
    static void Main(string[] args)
    {
        Student s1 = new Student(1, "Saurav");
        Student s2 = new Student(s1);

        s1.Display();
        s2.Display();
    }
}

```

✅ Output:

Roll No: 1, Name: Saurav

Roll No: 1, Name: Saurav

2.4 Static Constructor

```

class Example
{
    public static int count;

    static Example()
    {
        count = 10;
        Console.WriteLine("Static Constructor called");
    }

    public Example()
    {
        count++;
        Console.WriteLine("Object Created. Count = " + count);
    }
}

```

```

    }
}

class Program
{
    static void Main(string[] args)
    {
        Example e1 = new Example();
        Example e2 = new Example();
    }
}

```

✅ Output:

Static Constructor called
 Object Created. Count = 11
 Object Created. Count = 12

2.5 Private Constructor

```

class Example
{
    private Example()
    {
        Console.WriteLine("Private Constructor called");
    }

    public static Example CreateObject()
    {
        return new Example();
    }
}

class Program
{
    static void Main(string[] args)
    {
        Example obj = Example.CreateObject();
    }
}

```

✅ Output:

Private Constructor called